



Mage Labs DEX

Solana Smart Contract Security Audit Report

Client: Mage Labs
Auditor: Chain-Fox
Date: August 11, 2025
Version: 2.0.0

Security Audit Report

Prepared by: **Chain-Fox**

Table of Contents

1. [Executive Summary](#)
2. [Project Overview](#)
3. [Audit Methodology](#)
4. [Findings & Resolutions](#)

Executive Summary

Purpose:

This audit aims to thoroughly review the Mage Labs DEX Solana Smart Contracts for security, correctness, and best practices compliance.

Scope:

The audit covers all core contract instructions, state management, and security-critical logic. The initial audit was performed on the code version 2025-07-15. The second round of audit was performed on the code version 2025-08-01 with some code updates.

Methodology:

Combination of AI-assisted automatic & semi-automatic analysis, and manual code review was used.

Key Findings:

Several ownership checks missing on PDA accounts, missing parameter validation, missing input size validation, redundant code, and formatting issues.

Recommendations:

Add explicit ownership and parameter validation checks, enforce input size limits, remove redundant code, and maintain code formatting consistency.

Project Overview

Project Description:
Mage Labs DEX is a pioneering platform on the Solana blockchain that features advanced DEX technology with optimized Lamport dynamics.

- Technology Stack:**
- Rust version: 1.86.0
 - Frameworks/Libraries: Anchor, Solana SDK
 - Blockchain Platform: Solana

Code Structure:

```
mage
|- client
  |- src
    |- amm claim constants.rs health_check.rs main.rs network.rs
|- magelabs-ts
|- programs
  |- magelabs
    |- src
      |- constants.rs curve error.rs instructions lib.rs states tests utils
|- pubkey-env-macro
|- scripts
|- tests
```

Audit Methodology

- Automatic and Semi-auto Analysis:**
- Used Rust linters, cargo auditors, and custom static code inspection tools.
 - Used Kani to verify the core logic.
 - AI assisted code inspection and summarization.

Manual Review:
Thorough line-by-line manual review focusing on security-sensitive logic and state transitions.

Findings & Resolutions

Brief Summary:

Issue	Impact	Status
FIND-001: Improperly closing accounts	Medium	Resolved
FIND-002: Missing owner check on account authority	Medium	Resolved
FIND-003: Missing owner check on account pool_state	Medium	Resolved

Chain-Fox

FIND-004: Missing parameter validation on create_amm_config	Medium	Resolved
FIND-005: Missing parameter validation on add_vip_member	Low	Resolved
BAD-001: Ignoring the return of add_addresses & missing event emission	Low	Resolved
BAD-002: Redundant and erroneous `fn custom_create_associated_token_account`	Low	Resolved
BAD-003: Redundant status comparison	Low	Resolved
BAD-004: Redundant variables	Low	Resolved
BAD-005: Unhandled panic	Low	Resolved
BAD-006: Comments incomplete or ambiguous	Info	Resolved
BAD-007: Unformatted code	Info	Resolved

Detailed Analysis:

FIND-001: Improperly closing accounts (2025-07-15)

Improperly closing accounts without marking them as closed, enabling reuse/exploitation. `account\_to\_close` is not actually closed. Only the lamports are moved.

```
//programs/magelabs/src/instructions/close.rs:14, 23-26
#[derive(Accounts)]
pub struct Close<'info> {
    #[account(
        mut,
        address = crate::admin::ID @ ErrorCode::SignerIsNotAdmin
    )]
    pub admin: Signer<'info>,

    #[account(mut, owner = crate::ID)]
    pub account_to_close: UncheckedAccount<'info>,

    pub system_program: Program<'info, System>,
}

pub fn close(ctx: Context<Close>) -> Result<()> {
    let dest_starting_lamports = ctx.accounts.admin.lamports();
    let source_account_lamports = ctx.accounts.account_to_close.lamports();

    **ctx.accounts.admin.try_borrow_mut_lamports()? = dest_starting_lamports
        .checked_add(source_account_lamports)
        .ok_or(ErrorCode::ArithmeticError)?; // replace unwrap with error code.
    **ctx.accounts.account_to_close.try_borrow_mut_lamports()? = 0;

    // Zero out the account data
    let mut data = ctx.accounts.account_to_close.try_borrow_mut_data()?;
    for byte in data.iter_mut() {
        *byte = 0;
    }
}
```

Chain-Fox

```
}  
  
Ok()  
}
```

Resolution:

The close has been restricted to the account type `UserState`, and anchor's close constraint is used instead of manually closing. This issue has been resolved.

[FIND-002: Missing owner check on account authority \(2025-08-01\)](#)

``authority`` is a PDA account and the PDA address & bump are checked. But it misses the ownership check.

```
//programs/magelabs/src/instructions/initialize.rs:51  
pub struct Initialize<'info> {  
    /// Address paying to create the pool. Can be anyone  
    #[account(mut)]  
    pub creator: Signer<'info>,  
  
    /// Which config the pool belongs to.  
    pub amm_config: Box<Account<'info, AmmConfig>>,  
  
    /// CHECK: pool vault and lp mint authority  
    #[account(  
        seeds = [  
            AUTH_SEED.as_bytes(),  
        ],  
        bump,  
    )]  
    pub authority: UncheckedAccount<'info>,  
}
```

Resolution:

Added the following code in ``fn initialize`` to check the ownership of ``authority``

```
if ctx.accounts.authority.lamports() > 0 {  
    require!(  
        ctx.accounts.authority.owner == &crate::id(),  
        ErrorCode::InvalidAuthorityOwner  
    );  
}
```

Similar checks have been added to ``initialize_metadata``, ``collect_fund_fee``, ``collect_protocol_fee``, ``withdraw``, ``deposit``, ``claim_fee``, ``swap_base_output``, ``swap_base_input``.

[FIND-003: Missing owner check on account pool state \(2025-08-01\)](#)

``pool_state`` is a PDA account. Other PDA accounts (`vip_list`, etc.) use ``pool_state``'s pubkey as part of the seeds for account validation. There is no ownership check on ``pool_state``.

```
//programs/magelabs/src/instructions/admin/manage_allowlist.rs:26  
pub struct AddVipMember<'info> {  
    #[account(  
        mut,  
    )]  
}
```

Chain-Fox

```
        address = crate::admin::ID @ ErrorCode::SignerIsNotAdmin
    )]
    pub authority: Signer<'info>,

    #[account(
        init_if_needed,
        payer = authority,
        space = PoolVipList::max_space(),
        seeds = [
            VIP_LIST_SEED.as_bytes(),
            pool_state.key().as_ref()
        ],
        bump
    )]
    pub vip_list: Account<'info, PoolVipList>,

    pub pool_state: AccountLoader<'info, PoolState>,

    pub system_program: Program<'info, System>,
}
```

Resolution:

Added owner check annotation before `pool\_state`.

```
    #[account(
        owner = crate::id()
    )]
    pub pool_state: AccountLoader<'info, PoolState>,
```

Similar checks have been added to files `claim\_fee`, `deposit`, `Initialize\_metadata`, `swap\_base\_input`, `withdraw`, `collect\_fund\_fee`, `collect\_protocol\_fee`, `manage\_allow\_list`, `update\_pool\_status`.

This issue has been resolved.

[FIND-004: Missing parameter validation on create amm config \(2025-08-01\)](#)

`protocol\_fee` and `creator\_fee` are bounded in `update\_config` on
programs/magelabs/src/instructions/admin/update_config.rs:71

But they are not checked in `create\_config`

//programs/magelabs/src/instructions/admin/create_config.rs:54-55

```
pub fn create_amm_config(
    ctx: Context<CreateAmmConfig>,
    index: u64,
    protocol_fee: u64,
    creator_fee: u64,
    create_pool_fee: u64,
    base_fee_token_0: u64,
    base_fee_token_1: u64,
    fee_cap_divisor: u64,
) -> Result<()> {
    let amm_config = ctx.accounts.amm_config.deref_mut();
    amm_config.protocol_owner = ctx.accounts.owner.key();
    amm_config.bump = ctx.bumps.amm_config;
    amm_config.disable_create_pool = false;
    amm_config.index = index;
    amm_config.protocol_fee = protocol_fee;
    amm_config.creator_fee = creator_fee;
```

Chain-Fox

Resolution:

```
pub fn create_amm_config(
    ctx: Context<CreateAmmConfig>,
    index: u64,
    protocol_fee: u64,
    creator_fee: u64,
+   total_fee: u64,
    create_pool_fee: u64,
    base_fee_token_0: u64,
    base_fee_token_1: u64,
    fee_cap_divisor: u64,
) -> Result<()> {
+   // Validate total fee
+   require!(
+       total_fee <= 100_000, // 10% max total fee
+       ErrorCode::InvalidFeePercentage
+   );
}
```

Resolution:

The developer replaces `protocol\_fee` and `creator\_fee` with `total\_fee` and checks `total\_fee` is within 10%. This issue has been resolved.

[FIND-006: Missing parameter validation on add vip member \(2025-08-01\)](#)

There is no check on the length of the input `addresses`. `add\_addresses` will bound the number of addresses to be added but it will traverse every address. Therefore it still wastes computation when `addresses` vector is too large. The same issue is also found in `fn remove\_vip\_member`.

```
//programs/magelabs/src/instructions/admin/manage_allowlist.rs:31
pub fn add_vip_member(ctx: Context<AddVipMember>, addresses: Vec<Pubkey>) -> Result<()> {
    let vip_list = &mut ctx.accounts.vip_list;

    // Initialize if this is a new VIP list
    if vip_list.pool == Pubkey::default() {
        vip_list.pool = ctx.accounts.pool_state.key();
        vip_list.bump = ctx.bumps.vip_list;
    }

    // Add addresses (function returns true if any were added)
    vip_list.add_addresses(addresses);

    Ok(())
}
```

Resolution:

Limit the length of `addresses` to save computation at the start of the function. This issue has been resolved.

```
pub fn add_vip_member(ctx: Context<AddVipMember>, addresses: Vec<Pubkey>) -> Result<()> {
+   if addresses.len() > MAX_VIP_ADDRESSES {
+       return Err(ErrorCode::TooManyVipAddresses.into());
+   }
+
+   ...
}
```

Chain-Fox

[BAD-001: Ignoring the return of add_addresses & missing event emission \(2025-08-01\)](#)

`add\_addresses` returns a boolean to indicate whether the addition is successful or not. It is ignored in the code. No event is emitted. The same issue is also found in `fn remove\_vip\_member`.

Resolution:

Added event emission for the result of `add\_addresses` and `remove\_addresses`. This issue has been resolved.

```
pub fn add_vip_member(ctx: Context<AddVipMember>, addresses: Vec<Pubkey>) -> Result<()> {  
    ...  
+   let success = vip_list.add_addresses(addresses);  
+   let success_count = if success { addresses_count } else { 0 };  
+  
+   // BAD-001 audit fix: Emit event for VIP member operations  
+   emit!(VipMemberEvent {  
+       pool_id: ctx.accounts.pool_state.key(),  
+       authority: ctx.accounts.authority.key(),  
+       addresses: addresses_copy,  
+       operation: "add".to_string(),  
+       success_count,  
+       slot: Clock::get()?.slot,  
+   });  
+  
+   Ok::<(), Error>(())  
}
```

[BAD-002: Redundant and erroneous `fn custom\\_create\\_associated\\_token\\_account` \(2025-08-01\)](#)

Resolution:

Removed `fn custom\_create\_associated\_token\_account` in `//programs/magelabs/src/utis/token.rs`. This issue has been resolved.

[BAD-003: Redundant status comparison \(2025-08-01\)](#)

`status` is of type u8, min value is 0, and max value is 255. So `require\_gte!(255, status)` is always true and can be removed.

```
//programs/magelabs/src/instructions/admin/update_pool_status.rs:17  
pub fn update_pool_status(ctx: Context<UpdatePoolStatus>, status: u8) -> Result<()> {  
    require_gte!(255, status);  
}
```

Resolution:

Removed `require\_gte!(255, status)`. This issue has been resolved.

[BAD-004: Redundant variables \(2025-07-15\)](#)

`\_user` and `\_user\_last\_slot` are not used.

```
//programs/magelabs/src/instructions/withdraw.rs:115, 137-141  
pub fn withdraw(  
    ctx: Context<Withdraw>,  
    lp_token_amount: u64,  
    ...  
) -> Result<()> {  
    ...  
}
```


Chain-Fox

```
    minimum_token_0_amount: u64,  
    minimum_token_1_amount: u64,  
) -> Result<()> {  
    // ...  
    let _user = ctx.accounts.user.key();  
    // ...  
    let _user_last_slot = if user_state.last_snap_slot == 0 {  
        current_snap_slot  
    } else {  
        user_state.last_snap_slot  
    };  
};
```

Resolution:

The redundant code has been removed. The issue has been resolved.

[BAD-005: Unhandled panic \(2025-07-15\)](#)

Unwrap over-used in arithmetic operations in swap_base_input and swap_base_input.

e.g.

```
//programs/magelabs/src/instructions/swap_base_input.rs  
pool_state.swap_fees_token_0.checked_add(swap_fee).unwrap();
```

Resolution:

The unwrap has been replaced with ok_or(...). The issue has been resolved.

[BAD-006: Comments incomplete or ambiguous \(2025-07-15\)](#)

e.g.

```
//programs/magelabs/src/instructions/swap_base_input.rs
```

The comment before `fn swap\_base\_input`.

This comment claims that `you don't do anything with the swap\_fee`. However, the code explicitly calculates and stores all three fee components (`swap\_fee`, `creator\_fee`, and `protocol\_fee`) in the `pool\_state`.

Resolution:

The comment has been removed. The issue has been resolved.

[BAD-007: Unformatted code \(2025-08-01\)](#)

e.g.

```
//programs/magelabs/src/instructions/initialize_metadata.rs:24  
#[account(  
    seeds = [  
        AUTH_SEED.as_bytes(),  
    ],
```

Chain-Fox

```
    bump,  
  )  
  pub authority: UncheckedAccount<'info>,
```

Resolution:

Run `cargo fmt` before commit. This issue has been resolved.